8. 출력 파일 생성 및 코드 생성 모듈 분석 문서

개요

Circom 컴파일러의 최종 단계는 다양한 형태의 출력 파일을 생성하는 것입니다. 이 문서는 R1CS 제약 조건 파일, C++/WebAssembly 코드, 심볼 파일 등의 생성 과정을 상세히 분석합니다. 이러한 출력 파일들은 snarkjs, rapidSnark 등의 증명 시스템과 연동되어 실제 zkSNARK 증명을 생성하는 데 사용됩니다.

8.1 R1CS 파일 생성 과정 분석

ConstraintExporter 인터페이스

`constraint_writers` 모듈의 핵심은 `ConstraintExporter` 트레이트로, 다양한 형태의 출력 파일 생성을 위한 통합 인터페이스를 제공합니다:

```
pub trait ConstraintExporter {
fn r1cs(&self, out: &str, custom_gates: bool) -> Result<(), ()>;
fn json_constraints(&self, writer: &debug_writer::DebugWriter) -> Result<(), ()
fn sym(&self, out: &str) -> Result<(), ()>;
}
```

이 인터페이스는 세 가지 주요 출력 형태를 지원합니다:

- **R1CS 바이너리 파일**: 제약 조건의 효율적인 바이너리 표현
- **JSON 제약 조건**: 디버깅 및 분석을 위한 인간 친화적 형태

- **심볼 파일**: 신호 매핑 및 디버깅 정보

R1CS 파일 구조 및 바이너리 포맷

R1CS(Rank-1 Constraint System) 파일은 zkSNARK 증명 시스템의 핵심 입력 데이터로, 다음과 같은 구조를 가집니다:

파일 헤더 구조

```
const MAGIC: &[u8] = b"r1cs"; // 매직 넘버

const VERSION: &[u8] = &[1, 0, 0, 0]; // 버전 정보

const SECTIONS: u8 = 5; // 섹션 수
```

섹션 타입 정의

```
const HEADER_TYPE: &[u8] = &[1, 0, 0, 0]; // 헤더 섹션

const CONSTRAINT_TYPE: &[u8] = &[2, 0, 0, 0]; // 제약 조건 섹션

const WIRE2LABEL_TYPE: &[u8] = &[3, 0, 0, 0]; // 신호 매핑 섹션

const CUSTOM_GATES_USED_TYPE: &[u8] = &[4, 0, 0, 0]; // 사용된 커스텀 게이트

const CUSTOM_GATES_APPLIED_TYPE: &[u8] = &[5, 0, 0, 0]; // 적용된 커스텀 게이트
```

R1CSWriter 클래스 분석

`R1CSWriter` 는 R1CS 파일 생성의 핵심 클래스로, 섹션별 순차적 작성을 지원합니다:

```
pub  struct R1CSWriter {

field_size: usize,

writer: BufWriter<File>,
```

```
sections: [bool; SECTIONS as usize]
}
```

헤더 섹션 작성 과정

`HeaderSection` 은 회로의 기본 메타데이터를 포함합니다:

```
pub struct HeaderData {
pub field: BigInt, // 유한체 크기
pub total_wires: usize, // 전체 신호 수
pub public_outputs: usize, // 공개 출력 수
pub public_inputs: usize, // 공개 입력 수
pub private_inputs: usize, // 비밀 입력 수
pub number_of_labels: usize, // 라벨 수
pub number_of_constraints: usize, // 제약 조건 수
}
```

헤더 작성 과정:

1. **필드 크기 정보**: `bigint_as_bytes` 함수를 통해 유한체 크기를 바이너리로 변환
2. **회로 통계**: 신호 수, 제약 조건 수 등의 메타데이터를 순차적으로 기록
3. **크기 정보**: 각 데이터의 바이트 크기를 함께 저장하여 파싱 시 활용

제약 조건 섹션 작성

제약 조건은 R1CS 형태 `(A · witness) * (B · witness) = (C · witness)` 로 저장됩니다:

```
type Constraint = HashMap<usize, BigInt>;


impl ConstraintSection {
pub fn write_constraint_usize(
&mut self,
a: &Constraint,
b: &Constraint,
c: &Constraint,
) -> Result<(), ()>
}
```

각 제약 조건은 다음 형태로 저장됩니다:

- **선형 결합 블록**: `obtain_linear_combination_block` 함수가 생성
- **계수 정보**: 신호 인덱스와 해당 계수의 쌍으로 구성
- **압축 저장**: 0이 아닌 계수만 저장하여 공간 효율성 확보

신호 매핑 섹션

`SignalSection` 은 증인(witness) 인덱스와 신호 이름 간의 매핑을 저장합니다:

```
impl SignalSection {
pub fn write_signal_usize(&mut self, signal: usize) -> Result<(), ()> {
let (_, as_bytes) = BigInt::from(signal).to_bytes_le();
SignalSection::write_signal(self, &as_bytes)
}
```

```
}
```

커스텀 게이트 지원

Circom 2.0부터 지원되는 커스텀 게이트는 별도 섹션에 저장됩니다:

사용된 커스텀 게이트 정보

```
pub type CustomGatesUsedData = Vec<(String, Vec<BigInt>)>;

impl CustomGatesUsedSection {
pub fn write_custom_gates_usages(&mut self, data: CustomGatesUsedData) -> Res
}
```

적용된 커스텀 게이트 정보

```
pub type CustomGatesAppliedData = Vec<(usize, Vec<usize>)>;

impl CustomGatesAppliedSection {
pub fn write_custom_gates_applications(&mut self, data: CustomGatesAppliedDat
}
```

바이너리 포맷 최적화

R1CS 파일의 바이너리 포맷은 다음과 같은 최적화 기법을 사용합니다:

리틀 엔디안 인코딩

```
fn bigint_as_bytes(number: &BigInt, with_bytes: usize) -> (Vec<u8>, usize) {
let (_, value) = number.to_bytes_le();
into_format(&value, with_bytes)
}
```

섹션 크기 관리

```
fn initialize_section(writer: &mut BufWriter<File>, header: &[u8]) -> Result<u6
writer.write_all(header).map_err(|_err| {})?;
let go_back = writer.seek(SeekFrom::Current(0)).map_err(|_err| {})?;
writer.write_all(PLACE_HOLDER).map_err(|_| {})?;
Result::Ok(go_back)
}

fn end_section(writer: &mut BufWriter<File>, go_back: u64, size: usize) -> Resu
let go_back_1 = writer.seek(SeekFrom::Current(0)).map_err(|_err| {})?;
writer.seek(SeekFrom::Start(go_back)).map_err(|_err| {})?;
let (stream, _) = bigint_as_bytes(&BigInt::from(size), 8);
writer.write_all(&stream).map_err(|_err| {})?;
writer.seek(SeekFrom::Start(go_back_1)).map_err(|_err| {})?;
Result::Ok(())
```

```
}
```

이 방식을 통해:

1. **동적 크기 관리**: 섹션 작성 중 크기를 추적하고 나중에 헤더에 기록
2. **스트리밍 작성**: 메모리 사용량을 최소화하면서 대용량 파일 생성 지원
3. **무결성 보장**: 각 섹션의 크기 정보를 통한 파싱 검증 가능

snarkjs/rapidSnark 연동

생성된 R1CS 파일은 다음과 같이 증명 시스템과 연동됩니다:

snarkjs 연동 과정

1. **회로 로딩**: `snarkjs.r1cs.info(r1csFile)` - R1CS 파일 정보 확인
2. **키 생성**: `snarkjs.groth16.setup(r1csFile, ptauFile)` - 증명/검증 키 생성
3. **증명 생성**: `snarkjs.groth16.fullProve(input, wasmFile, zkeyFile)` - 증명 생성

rapidSnark 연동 과정

1. **C++ 컴파일**: 생성된 C++ 코드를 rapidSnark와 함께 컴파일
2. **고성능 증명**: 네이티브 코드를 통한 빠른 증명 생성
3. **메모리 최적화**: 대규모 회로에서의 효율적인 메모리 관리

파일 포맷 호환성

R1CS 파일 포맷은 다음과 같은 호환성을 보장합니다:

버전 관리

- **하위 호환성**: 이전 버전 파일 형식 지원
- **확장성**: 새로운 섹션 타입 추가 가능

- **검증**: 매직 넘버와 버전 정보를 통한 파일 유효성 검사

크로스 플랫폼 지원

- **엔디안 독립성**: 리틀 엔디안 고정 사용
- **정밀도 보장**: BigInt를 통한 임의 정밀도 산술 연산
- **표준 준수**: zkSNARK 생태계 표준 포맷 준수

이러한 R1CS 파일 생성 과정을 통해 Circom 컴파일러는 다양한 증명 시스템과의 원활한 연동을 보장하며, 효율적이고 안정적인 zkSNARK 증명 생성을 지원합니다.

8

.2 C++ 및 WebAssembly 코드 생성 분석

code_producers 모듈 아키텍처

`code_producers` 모듈은 Circom 컴파일러의 백엔드 역할을 담당하며, 두 가지 주요 타겟 플랫폼을 지원합니다:

모듈 구조

```
pub mod c_elements; // C++ 코드 생성
pub mod wasm_elements; // WebAssembly 코드 생성
pub mod components; // 공통 데이터 구조
```

C++ 코드 생성 과정 분석

CProducer 클래스 구조

`CProducer` 는 C++ 코드 생성의 핵심 클래스로, 회로 정보와 코드 생성 메타데이터를 관리합니다:

```rust
pub struct CProducer {
pub main_header: String, // 메인 템플릿 이름
pub main_is_parallel: bool, // 병렬 처리 지원 여부
pub has_parallelism: bool, // 전체 병렬성 사용 여부
pub number_of_main_outputs: usize, // 메인 출력 수
pub main_signal_offset: usize, // 신호 오프셋
pub number_of_main_inputs: usize, // 메인 입력 수
pub signals_in_witness: usize, // 증인 신호 수
pub total_number_of_signals: usize, // 전체 신호 수
pub number_of_components: usize, // 컴포넌트 수
pub size_of_component_tree: usize, // 컴포넌트 트리 크기
pub size_32_bit: usize, // 32비트 크기 (필드 의존적)
pub prime: String, // 유한체 소수
pub prime_str: String, // 필드 타입 문자열
pub main_input_list: InputList, // 메인 입력 리스트
pub witness_to_signal_list: SignalList, // 증인-신호 매핑
pub io_map: TemplateInstanceIOMap, // 템플릿 I/O 매핑
pub template_instance_list: TemplateListInfo, // 템플릿 인스턴스 정보
pub message_list: MessageList, // 메시지 리스트
pub field_tracking: Vec<String>, // 필드 상수 추적
pub busid_field_info: FieldMap, // 버스 필드 정보
pub no_asm: bool, // 어셈블리 최적화 비활성화
```

```
}
```

C++ 코드 생성 템플릿 시스템

C++ 코드 생성은 템플릿 기반 시스템을 사용하여 다양한 필드 타입을 지원합니다:

필드별 특화 코드

```
// 지원되는 필드 타입들
- bn128/ // BN128 곡선
- bls12381/ // BLS12-381 곡선
- bls12377/ // BLS12-377 곡선
- goldilocks/ // Goldilocks 필드
- grumpkin/ // Grumpkin 곡선
- pallas/ // Pallas 곡선
- vesta/ // Vesta 곡선
- secq256r1/ // SECQ256R1 곡선
- generic/ // 일반적인 필드
```

핵심 C++ 타입 정의

```
const T_U64: &str = "u64";
const T_U32: &str = "u32";
const T_U8: &str = "u8";
const T_P_FR_ELEMENT: &str = "PFrElement";
const T_FR_ELEMENT: &str = "FrElement";
```

주요 데이터 구조

```
const S_CIRCOM_HASH_ENTRY: &str = "Circom_HashEntry";

const S_CIRCOM_COMPONENT: &str = "Circom_Component";

const S_IOFIELD_DEF: &str = "IOFieldDef";
```

produce_c_code 함수 구현 방법

C++ 코드 생성은 다음과 같은 단계로 진행됩니다:

1. 템플릿 헤더 생성

```
pub fn collect_template_headers(producer: &CProducer, instances: &TemplateList

let mut template_headers = vec![];

for instance in instances {

let params_run = vec![declare_ctx_index(), declare_circom_calc_wit()];

let params_create = vec![

declare_signal_offset(),

declare_component_offset(),

declare_circom_calc_wit(),

declare_component_name(),

declare_component_father(),

];

if instance.is_parallel {

let run_header = format!("void {}_run_parallel({});", instance.name, params_run

let create_header = format!("void {}_create_parallel({});", instance.name, para
```

```
template_headers.push(create_header);
template_headers.push(run_header);
}
if instance.is_not_parallel {
let run_header = format!("void {}_run({});", instance.name, params_run);
let create_header = format!("void {}_create({});", instance.name, params_create
template_headers.push(create_header);
template_headers.push(run_header);
}
}
template_headers
}
```

2. 함수 헤더 생성

```
pub fn collect_function_headers(producer: &CProducer, functions: Vec<String>)
let mut function_headers = vec![];
for function in functions {
let params = vec![
declare_circom_calc_wit(),
if producer.prime_str != "goldilocks" { declare_lvar_pointer()
} else { declare_64bit_lvar_array() },
declare_component_father(),
```

```
if producer.prime_str != "goldilocks" { declare_dest_pointer()
} else { declare_64bit_dest_reference() },
declare_dest_size(),
];
let header = format!("void {}({});", function, argument_list(params));
function_headers.push(header);
}
function_headers
}
```

3. 데이터 파일(.dat) 생성

```
pub fn generate_dat_file(dat_file: &mut dyn Write, producer: &CProducer) -> s
// 해시맵 생성 및 저장
let aux = producer.get_main_input_list();
let map = generate_hash_map(&aux, producer.get_input_hash_map_entry_size());
let hashmap = generate_dat_from_hash_map(&map);
dat_file.write_all(&hashmap)?;
// 증인-신호 리스트 저장
let s = generate_dat_witness_to_signal_list(producer.get_witness_to_signal_list
dat_file.write_all(&s)?;
// 상수 리스트 저장 (Goldilocks 제외)
if producer.prime_str != "goldilocks" {
let s = generate_dat_constant_list(producer, producer.get_field_constant_list()
```

```
dat_file.write_all(&s)?;
}
// I/O 매핑 정보 저장
let iomap = generate_dat_io_signals_info(&producer, producer.get_io_map());
dat_file.write_all(&iomap)?;
// 버스 필드 정보 저장
if producer.get_io_map().len() > 0 {
let fieldmap = generate_dat_bus_field_info(&producer, producer.get_busid_field_
dat_file.write_all(&fieldmap)?;
}
dat_file.flush()?;
Ok(())
}
```

WebAssembly 코드 생성 분석

WASMProducer 클래스 구조

`WASMProducer` 는 WebAssembly 코드 생성을 위한 특화된 구조를 가집니다:

```
pub struct WASMProducer {
pub main_signal_offset: usize,
pub fr_memory_size: usize, // 필드 크기에 따른 메모리 크기
pub size_32_bit: usize,
```

```
pub wat_flag: bool, // WAT 형식 출력 여부
// ... 기본 회로 정보들 ...
// WebAssembly 특화 태그들
offset_tag: String, // "$offset"
signal_offset_tag: String, // "$signaloffset"
signal_start_tag: String, // "$signalstart"
sub_cmp_tag: String, // "$subcmp"
io_info_tag: String, // "$ioinfo"
result_address_tag: String, // "$resultaddress"
cstack_tag: String, // "$cstack"
lvar_tag: String, // "$lvar"
expaux_tag: String, // "$expaux"
// ... 기타 태그들 ...
}
```

produce_wasm_code 함수 구현

WebAssembly 코드 생성은 다음과 같은 구조로 진행됩니다:

1. WASM 모듈 구조 생성

```
impl WriteWasm for Circuit {
fn produce_wasm(&self, producer: &WASMProducer) -> Vec<String> {
let mut code = vec![];
code.push("(module".to_string());
// 임포트 리스트 생성
```

```
let mut code_aux = generate_imports_list();
code.append(&mut code_aux);
// 메모리 정의 생성
code_aux = generate_memory_def_list(&producer);
code.append(&mut code_aux);
// 필드 타입 정의
code_aux = fr_types(&producer.prime_str);
code.append(&mut code_aux);
// 타입 리스트 생성
code_aux = generate_types_list();
code.append(&mut code_aux);
// 익스포트 리스트 생성
code_aux = generate_exports_list();
code.append(&mut code_aux);
// 필드 연산 코드
code_aux = fr_code(&producer.prime_str);
code.append(&mut code_aux);
// 실제 프로그램 코드
for f in &self.functions {
code.append(&mut f.produce_wasm(producer));
}
for t in &self.templates {
code.append(&mut t.produce_wasm(producer));
```

```
}
code.push(")".to_string());
code
}
}
```

2. WASM 명령어 생성 함수들

```
pub fn set_constant(value: &str) -> WasmInstruction {
format!("i32.const {}", value)
}

pub fn get_local(value: &str) -> WasmInstruction {
format!("local.get {}", value)
}

pub fn set_local(value: &str) -> WasmInstruction {
format!("local.set {}", value)
}

pub fn add32() -> WasmInstruction {
"i32.add".to_string()
}
```

```
pub fn load32(offset: Option<&str>) -> WasmInstruction {
let code = "i32.load".to_string();
if let Option::Some(o) = offset {
format!("{} offset={}", code, o)
} else {
code
}
}
```

3. 메모리 레이아웃 관리

```
impl WASMProducer {
pub fn get_raw_prime_start(&self) -> usize {
4 + self.fr_memory_size
}
pub fn get_shared_rw_memory_start(&self) -> usize {
(4 * self.size_32_bit) + 8 + self.get_raw_prime_start()
}
pub fn get_input_signals_hashmap_start(&self) -> usize {
(4 * self.size_32_bit) + 8 + self.get_shared_rw_memory_start()
}
pub fn get_signal_memory_start(&self) -> usize {
self.get_signal_free_pos() + 4
```

```
}
pub fn get_component_tree_start(&self) -> usize {
self.get_component_free_pos() + 4
}
}
```

생성된 코드의 구조 및 연동 방법

C++ 코드 구조

생성된 C++ 코드는 다음과 같은 구조를 가집니다:

1. 헤더 파일 구조

```
// 필드 연산 함수들
void Fr_add(PFrElement r, PFrElement a, PFrElement b);
void Fr_mul(PFrElement r, PFrElement a, PFrElement b);
void Fr_copy(PFrElement r, PFrElement a);

// 템플릿 함수들
void TemplateName_create(uint soffset, uint coffset, Circom_CalcWit* ctx,
std::string componentName, uint componentFather);
void TemplateName_run(uint ctx_index, Circom_CalcWit* ctx);

// 병렬 처리 버전
void TemplateName_create_parallel(/* 동일한 매개변수 */);
```

```
void TemplateName_run_parallel(uint ctx_index, Circom_CalcWit* ctx);
```

2. 메인 실행 구조

```
class Circom_CalcWit {
public:
FrElement* signalValues;
Circom_Component* componentMemory;
std::map<u32,IOFieldDefPair> templateInsId2IOSignalInfo;
// ... 기타 멤버들 ...
void calculateWitness(std::map<std::string, std::vector<BigInt>>& input);
};
```

WebAssembly 코드 구조

생성된 WASM 코드는 다음과 같은 구조를 가집니다:

1. 모듈 인터페이스

```
(module
(import "runtime" "exceptionHandler" (func $exceptionHandler (param i32)))
(import "runtime" "printErrorMessage" (func $printErrorMessage))
(import "runtime" "writeBufferMessage" (func $writeBufferMessage))
(memory (export "memory") 2000)
(export "init" (func $init))
(export "setInputSignal" (func $setInputSignal))
```

```
(export "getWitness" (func $getWitness))
(export "getInputSize" (func $getInputSize))
// ... 기타 익스포트들 ...
)
```

2. 메모리 레이아웃

```
메모리 주소 0: 버전 정보
메모리 주소 4: 필드 크기 정보
메모리 주소 8: 공유 읽기/쓰기 메모리 시작
메모리 주소 X: 입력 신호 해시맵
메모리 주소 Y: 신호 메모리
메모리 주소 Z: 컴포넌트 트리
```

snarkjs/rapidSnark 연동 방법

snarkjs 연동 과정

1. JavaScript 환경에서의 사용

```
const circomlib = require("circomlib");
const snarkjs = require("snarkjs");

// WASM 파일 로딩
const wasm = await circomlib.WitnessCalculatorBuilder(wasmBuffer);
```

```
// 입력 설정 및 증인 계산
const input = { "in1": 123, "in2": 456 };
const witness = await wasm.calculateWitness(input);

// 증명 생성
const { proof, publicSignals } = await snarkjs.groth16.fullProve(
input, wasmFile, zkeyFile
);
```

2. 브라우저 환경 지원

- **WebAssembly 최적화**: 브라우저에서 직접 실행 가능
- **메모리 효율성**: 스트리밍 방식의 증인 계산
- **비동기 처리**: Promise 기반 인터페이스

rapidSnark 연동 과정

1. C++ 컴파일 과정

```
# 생성된 C++ 코드 컴파일
g++ -O3 -DUSE_ASM -fPIC -std=c++17 \
circuit.cpp main.cpp \
-lgmp -lsodium -lcrypto \
-o circuit_prover
```

2. 고성능 증명 생성

```
#include "circom.hpp"
#include "calcwit.hpp"

int main() {
// 회로 초기화
Circom_CalcWit calcWit(datFile);
// 입력 설정
std::map<std::string, std::vector<BigInt>> input;
input["in1"] = {BigInt("123")};
input["in2"] = {BigInt("456")};
// 증인 계산
calcWit.calculateWitness(input);
// 증명 생성 (rapidSnark 라이브러리 사용)
auto proof = generateProof(calcWit.witness, zkeyFile);
return 0;
}
```

3. 성능 최적화 기법

- **어셈블리 최적화**: 필드 연산에 대한 인라인 어셈블리 사용
- **병렬 처리**: OpenMP를 통한 멀티스레드 지원
- **메모리 풀링**: 대규모 회로를 위한 효율적인 메모리 관리
- **SIMD 최적화**: 벡터 연산을 통한 성능 향상

코드 생성 최적화 기법

필드별 특화 최적화

1. Goldilocks 필드 최적화

```
if producer.prime_str == "goldilocks" {
// 64비트 네이티브 연산 사용
declare_64bit_lvar_array()
} else {
// 일반적인 BigInt 연산 사용
declare_lvar_pointer()
}
```

2. BN128 곡선 최적화

- **Montgomery 형태**: 효율적인 모듈러 연산
- **어셈블리 최적화**: x86-64 특화 필드 연산
- **배치 처리**: 다중 연산의 일괄 처리

템플릿 시스템 최적화

1. 병렬 처리 지원

```
if instance.is_parallel {
// 병렬 처리 버전 생성
let run_header = format!("void {}_run_parallel({});", instance.name, params_run
let create_header = format!("void {}_create_parallel({});", instance.name, para
}
```

```
if  instance.is_not_parallel {
// 순차 처리 버전 생성
let  run_header = format!("void {}_run({});", instance.name, params_run);
let  create_header = format!("void {}_create({});", instance.name, params_create
}
```

2. 커스텀 게이트 지원

```
if  instance.is_extern_c {
// 외부 C 함수 인터페이스 생성
let  run_header = format!("void {}({});", instance.template_name, argument_list(
println!("*** The method {} generated by the custom gate {} needs to be implemen
run_header, instance.template_name);
}
```

이러한 코드 생성 과정을 통해 Circom 컴파일러는 다양한 플랫폼과 성능 요구사항에 맞는 최적화된 코드를 생성하며, zkSNARK 생태계의 다양한 도구들과 원활한 연동을 지원합니다.##

8.3 심볼 파일 및 디버깅 정보 생성 분석

.sym 파일 구조 및 신호 매핑 정보

심볼 파일(.sym)은 Circom 컴파일러가 생성하는 중요한 디버깅 정보 파일로, 증인(witness) 인덱스와 실제 신호 이름 간의 매핑을 제공합니다.

SymElem 구조체 분석

```rust
pub struct SymElem {
pub original: i64, // 원본 신호 ID
pub witness: i64, // 증인 배열에서의 인덱스
pub node_id: i64, // AST 노드 ID
pub symbol: String, // 신호의 실제 이름
}

impl ToString for SymElem {
fn to_string(&self) -> String {
format!("{},{},{},{}", self.original, self.witness, self.node_id, self.symbol)
}
}
```

각 `SymElem` 은 다음 정보를 포함합니다:

- **original**: 컴파일 과정에서의 원본 신호 식별자
- **witness**: 최종 증인 배열에서의 위치 (0부터 시작)
- **node_id**: 소스 코드의 AST 노드와의 연결점
- **symbol**: 사용자가 정의한 실제 신호 이름 (예: "main.in[0]", "multiplier.out")

.sym 파일 생성 과정

```rust
pub struct SymFile {
writer: BufWriter<File>,
}
```

```rust
impl SymFile {
pub fn new(file: &str) -> Result<SymFile, ()> {
let file = File::create(file).map_err(|_err| {})?;
let writer = BufWriter::new(file);
Result::Ok(SymFile { writer })
}

pub fn write_sym_elem(sym: &mut SymFile, elem: SymElem) -> Result<(), ()> {
sym.writer.write_all(elem.to_string().as_bytes()).map_err(|_err| {})?;
sym.writer.write_all(b"\n").map_err(|_err| {})
}
pub fn finish_writing(mut sym: SymFile) -> Result<(), ()> {
sym.writer.flush().map_err(|_err| {})
}
}
```

.sym 파일 포맷 예시

```
0,0,1,one
1,1,15,main.in1
2,2,16,main.in2
3,3,25,main.out
```

```
4,4,35,multiplier.a

5,5,36,multiplier.b

6,6,45,multiplier.c
```

각 라인은 `original,witness,node_id,symbol` 형태로 구성되며:

- 첫 번째 라인 `0,0,1,one` 은 항상 상수 1을 나타냄
- 이후 라인들은 실제 회로의 신호들을 순서대로 나열
- 신호 이름은 계층적 구조를 반영 (예: `main.multiplier[0].in`)

디버깅을 위한 메타데이터 생성 및 활용

DebugWriter 클래스 분석

```rust
#[derive(Clone)]
pub struct DebugWriter {
pub json_constraints: String,
}

impl DebugWriter {
pub fn new(c: String) -> Result<DebugWriter, ()> {
Result::Ok(DebugWriter { json_constraints: c })
}

pub fn build_constraints_file(&self) -> Result<ConstraintJSON, ()> {
ConstraintJSON::new(&self.json_constraints)
```

```
}

}
```

`DebugWriter` 는 제약 조건을 JSON 형태로 출력하여 디버깅을 지원합니다.

Log 구조체를 통한 컴파일 통계

```
pub struct Log {
pub no_linear: usize, // 선형 제약 조건 수
pub no_non_linear: usize, // 비선형 제약 조건 수
pub no_labels: usize, // 라벨 수
pub no_wires: usize, // 전체 신호(와이어) 수
pub no_public_inputs: usize, // 공개 입력 수
pub no_private_inputs: usize, // 비밀 입력 수
pub no_private_inputs_witness: usize, // 증인에 포함된 비밀 입력 수
pub no_public_outputs: usize, // 공개 출력 수
}

impl Log {
pub fn print(log: &Log) {
println!("non-linear constraints: {}", log.no_non_linear);
println!("linear constraints: {}", log.no_linear);
println!("public inputs: {}", log.no_public_inputs);
if log.no_private_inputs == log.no_private_inputs_witness {
println!("private inputs: {}", log.no_private_inputs);
```

```
} else if log.no_private_inputs_witness == 0 {
println!("private inputs: {} (none belong to witness)", log.no_private_inputs);
} else {
println!("private inputs: {} ({} belong to witness)",
log.no_private_inputs, log.no_private_inputs_witness);
}
println!("public outputs: {}", log.no_public_outputs);
println!("wires: {}", log.no_wires);
println!("labels: {}", log.no_labels);
}
}
```

이 통계 정보는 다음과 같은 디버깅 목적으로 활용됩니다:

- **회로 복잡도 분석**: 제약 조건 수를 통한 성능 예측
- **신호 분류 확인**: 입력/출력 신호의 올바른 분류 검증
- **최적화 효과 측정**: 컴파일 전후의 제약 조건 수 비교

JSON 출력 포맷 및 대체 정보 로깅 시스템

ConstraintJSON 클래스 분석

```
pub struct ConstraintJSON {
writer_constraints: BufWriter<File>,
constraints_flag: bool,
}
```

```
impl ConstraintJSON {
pub fn new(file: &str) -> Result<ConstraintJSON, ()> {
let file_constraints = File::create(file).map_err(|_err| {})?;
let mut writer_constraints = BufWriter::new(file_constraints);


writer_constraints.write_all(b"{").map_err(|_err| {})?;
writer_constraints.write_all(b"\n\"constraints\": ["").map_err(|_err| {})?;
Result::Ok(ConstraintJSON { writer_constraints, constraints_flag: false })
}
pub fn write_constraint(&mut self, constraint: &str) -> Result<(), ()> {
if !self.constraints_flag {
self.constraints_flag = true;
self.writer_constraints.write_all(b"\n").map_err(|_err| {})?;
} else {
self.writer_constraints.write_all(b",\n").map_err(|_err| {})?;
}
self.writer_constraints.write_all(constraint.as_bytes()).map_err(|_err| {})?;
Result::Ok(())
}
pub fn end(mut self) -> Result<(), ()> {
self.writer_constraints.write_all(b"\n]\n}").map_err(|_err| {})?;
Result::Ok(())
```

```
}
}
```

JSON 제약 조건 파일 구조

```
{
"constraints": [
{
"a": {"1": "1", "2": "0"},
"b": {"3": "1"},
"c": {"4": "1"}
},
{
"a": {"1": "1"},
"b": {"1": "1"},
"c": {"5": "1"}
}
]
}
```

각 제약 조건은 R1CS 형태 `A·witness * B·witness = C·witness` 로 표현되며:

- **a, b, c**: 각각 A, B, C 벡터의 비영 원소들
- **키**: 신호(증인) 인덱스
- **값**: 해당 신호의 계수

SignalsJSON 클래스 분석

```
pub struct SignalsJSON {
writer_signals: BufWriter<File>,
}

impl SignalsJSON {
pub fn new(file: &str) -> Result<SignalsJSON, ()> {
let file_signals = File::create(file).map_err(|_err| {})?;
let mut writer_signals = BufWriter::new(file_signals);
writer_signals.write_all(b"{").map_err(|_err| {})?;
writer_signals.write_all(b"\n\"signalName2Idx\": {").map_err(|_err| {})?;
writer_signals.write_all(b"\n\"one\" : \"0\"").map_err(|_err| {})?;
Result::Ok(SignalsJSON { writer_signals })
}
pub fn write_correspondence(&mut self, signal: String, data: String) -> Resul
self.writer_signals
.write_all(format!(",\n\"{}\" : {}", signal, data).as_bytes())
.map_err(|_err| {})
}
}
```

신호 매핑 JSON 파일 구조

```
{
"signalName2Idx": {
"one": "0",
"main.in1": "1",
"main.in2": "2",
"main.out": "3",
"main.multiplier.a": "4",
"main.multiplier.b": "5",
"main.multiplier.c": "6"
}
}
```

SubstitutionJSON 클래스 분석

```
pub struct SubstitutionJSON {
writer_substitutions: BufWriter<File>,
first: bool,
}

impl SubstitutionJSON {
pub fn write_substitution(&mut self, signal: &str, substitution: &str) -> Res
if self.first {
self.first = false;
```

```
self.writer_substitutions.write_all(b"\n").map_err(|_err| {})?;
} else {
self.writer_substitutions.write_all(b",\n").map_err(|_err| {})?;
}
let  substitution = format!("\"{}\" : {}", signal, substitution);
self.writer_substitutions.write_all(substitution.as_bytes()).map_err(|_err| {})?
Result::Ok(())
}
}
```

신호 치환 정보 JSON 구조

```
{
"main.temp":  "main.in1 * main.in2",
"main.intermediate":  "main.temp + 1",
"main.out":  "main.intermediate"
}
```

R1CS 파일 읽기 및 검증 시스템

R1CSReader 클래스 분석

디버깅 정보의 완전성을 보장하기 위해 `R1CSReader` 는 생성된 R1CS 파일을 다시 읽어 검증하는 기능을 제공합니다:

```rust
pub struct R1CSReader {
field_size: usize,
reader: BufReader<File>,
sections: [bool; SECTIONS as usize],
}

pub enum R1CSParsingError {
InvalidMagicNumber,
InvalidVersion,
InvalidSectionHeader,
InvalidSectionType,
SectionNotPresent(String),
}
```

파일 무결성 검증 과정

```rust
fn read_initialization(r1cs: &mut R1CSReader) -> Result<Vec<u8>, std::io::Error
// 매직 넘버 검증
let mut buffer = vec![0; MAGIC.len()];
r1cs.reader.read_exact(&mut buffer)?;
if buffer != MAGIC {
return Err(std::io::Error::new(std::io::ErrorKind::Other,
R1CSParsingError::InvalidMagicNumber.to_string()));
```

```
}
// 버전 정보 검증
let mut buffer = vec![0; VERSION.len()];
r1cs.reader.read_exact(&mut buffer)?;
if buffer != VERSION {
return Err(std::io::Error::new(std::io::ErrorKind::Other,
R1CSParsingError::InvalidVersion.to_string()));
}
Ok(buffer)
}
```

섹션별 데이터 검증

```
pub fn read_r1cs(input: &str) -> Result<R1CSData, std::io::Error> {
let mut info = R1CSData::new();
let mut r1cs = R1CSReader::new(input.to_string())?;

// 초기화 및 섹션 맵 구축
let buffer = read_initialization(&mut r1cs)?;
let n_sections = buffer[0] as usize;
info.custom_gates = n_sections == 5;
// 각 섹션의 시작 위치 계산
let mut section_starts = HashMap::new();
```

```
for _ in 0..n_sections {
let section_type = read_section_type(&mut r1cs)?;
let section_size = read_section_size(&mut r1cs)?;
section_starts.insert(section_type, current_offset);
}
// 섹션별 데이터 읽기 및 검증
read_sections(&mut info, r1cs, section_starts)?;
Ok(info)
}
```

디버깅 정보 활용 방법

1. 증명 실패 시 디버깅

심볼 파일을 활용한 디버깅 과정:

```
# 증명 생성 실패 시
snarkjs groth16 prove circuit.zkey witness.wtns proof.json public.json

# 실패한 제약 조건 확인
circom circuit.circom --r1cs --sym --json
# 생성된 파일들:
# - circuit.r1cs: 바이너리 제약 조건
# - circuit.sym: 신호 매핑 정보
# - constraints.json: 인간 친화적 제약 조건
```

```
# - signals.json: 신호 이름 매핑
```

2. 성능 분석 및 최적화

로그 정보를 통한 성능 분석:

```
// 컴파일 통계 출력 예시
non-linear constraints: 1250
linear constraints: 340
public inputs: 2
private inputs: 15 (12 belong to witness)
public outputs: 1
wires: 1608
labels: 1608
```

이 정보를 통해:

- **제약 조건 수**: 증명 생성 시간과 직접적 연관
- **신호 수**: 메모리 사용량 예측
- **입출력 분류**: 인터페이스 설계 검증

3. 회로 검증 및 감사

JSON 출력을 통한 상세 분석:

```
// constraints.json 파싱을 통한 제약 조건 분석
const constraints = JSON.parse(fs.readFileSync('constraints.json'));
const signals = JSON.parse(fs.readFileSync('signals.json'));
```

```
// 특정 신호와 관련된 제약 조건 찾기
function findConstraintsForSignal(signalName) {
const signalIdx = signals.signalName2Idx[signalName];
return constraints.constraints.filter(c =>
c.a[signalIdx] || c.b[signalIdx] || c.c[signalIdx]
);
}

// 제약 조건 복잡도 분석
function analyzeConstraintComplexity() {
return constraints.constraints.map(c => ({
aTerms: Object.keys(c.a).length,
bTerms: Object.keys(c.b).length,
cTerms: Object.keys(c.c).length
}));
}
```

4. 자동화된 테스트 및 검증

심볼 파일을 활용한 자동 테스트:

```
# Python을 이용한 증인 검증 스크립트
def verify_witness_with_symbols(witness_file, sym_file, expected_values):
```

```
# .sym 파일 파싱

signal_map = {}

with  open(sym_file, 'r') as f:

for line in f:

original, witness_idx, node_id, symbol = line.strip().split(',')

signal_map[symbol] = int(witness_idx)

# 증인 파일 읽기

with  open(witness_file, 'r') as f:

witness = json.load(f)

# 예상 값과 실제 값 비교

for signal_name, expected_value in expected_values.items():

if signal_name in signal_map:

witness_idx = signal_map[signal_name]

actual_value = witness[witness_idx]

assert actual_value == expected_value, \

f"Signal {signal_name}: expected {expected_value}, got {actual_value}"
```

디버깅 정보의 보안 고려사항

1. 민감한 정보 노출 방지

심볼 파일에는 회로의 내부 구조가 노출되므로:

- **프로덕션 환경**: 심볼 파일 생성 비활성화 옵션 제공
- **개발 환경**: 상세한 디버깅 정보 활성화
- **감사 환경**: 제한적 정보만 포함하는 축약된 심볼 파일

2. 파일 무결성 보장

```rust
// 체크섬을 통한 파일 무결성 검증
use sha2::{Sha256, Digest};

fn verify_file_integrity(file_path: &str, expected_hash: &str) -> bool {
let mut file = File::open(file_path).unwrap();
let mut hasher = Sha256::new();
std::io::copy(&mut file, &mut hasher).unwrap();
let result = hasher.finalize();
format!("{:x}", result) == expected_hash
}
```

이러한 포괄적인 디버깅 정보 시스템을 통해 Circom 컴파일러는 개발자가 복잡한 zkSNARK 회로를 효율적으로 개발, 디버깅, 최적화할 수 있도록 지원하며, 동시에 프로덕션 환경에서의 보안성도 보장합니다.